# ζ Riemann_Lab

## Guide — Distribution PC1+PC2 et Dashboard Live

| Machine | Rôle | CPU | Vitesse |
|---|---|---|---|
| PC1 · zeta-lab | Orchestrateur + calcul principal | i7 4c / 3.4 GHz | 624 z/s |
| PC2 · zeta-calc-sec | Second nœud de calcul (SSH) | E8400 2c / 3.0 GHz | 52 z/s |
| PC3 · zeta-backup | Backup / Monitoring | E2140 2c | — |
| PC4 · zeta-secure | Bastion VPN / WireGuard | Pentium 4 | — |

**Auteur :** hprzeta · **Branche :** Riemann_Lab_C · **Commit source :** 30d05ee · **Date :** 17 juin 2026

## 1. Objectif et contexte

Ce guide documente le pipeline de **distribution multi-machines** du projet Riemann_Lab : comment répartir le calcul des zéros de ζ(s) entre PC1 et PC2, comment superviser l'avancement en direct via le dashboard **ZETA RUN PROGRESS**, et comment interpréter les résultats et les logs.

**Algorithme :** v13 — compute_zeros_v13.py avec STEP adaptatif corrigé (0.00497 au lieu de 0.010 fixe). Le bug du STEP causait 6 zéros manquants à grand T (T ≈ 384 798) ; le correctif a été validé et commité en 30d05ee.

## 2. Architecture de la distribution

Le script **scripts/zeta_distribute.py** répartit le calcul selon la formule de Riemann–von Mangoldt :

$$N(T) \approx (T/2\pi) \cdot \ln(T/2\pi e) + S(T) + O(1/T)$$

**Calcul du pivot T_PIVOT :**
Le pivot est la valeur de T qui divise le travail de façon à ce que PC1 et PC2 terminent **exactement au même moment**. L'équation d'équilibre est :

$$N(T_PIVOT) / V_PC1 = [N(T_MAX) - N(T_PIVOT)] / V_PC2$$

Avec V_PC1 = 624 z/s et V_PC2 = 52 z/s (rapport ×12), PC1 reçoit **93,5 % des zéros** (N ≈ 764 903) et PC2 seulement **6,5 %** (N ≈ 53 510). Une marge de –15 % est appliquée à PC2 pour compenser l'overhead SSH.

**Étapes du pipeline zeta_distribute.py :**

1. Calcul du pivot T_PIVOT et de la segmentation.
2. Activation de zeta_turbo_on.sh sur PC1 (governor performance).
3. Lancement parallèle : compute_zeros_v13.py en subprocess local (PC1) + SSH (PC2).
4. Lancement du dashboard dans la session tmux zeta-progress.
5. Attente de la fin des deux processus.
6. Récupération du CSV PC2 par scp.
7. Fusion des deux CSV + déduplication (tolérance 0.01).
8. Validation Turing-Backlund globale sur [14, T_MAX].
9. Génération du rapport + désactivation du turbo.

## 3. Procédure de lancement

**Étape 1 — Dans Claude Code (l'application Anthropic) :**

```
Lance le run T=500000 distribué PC1+PC2 avec le STEP corrigé.

Utilise : nohup python scripts/zeta_distribute.py 500000 2>&1 &

Assure-toi que zeta_run_progress.py se lance dans une session tmux
zeta-progress.

Donne-moi la commande exacte pour voir le dashboard.
```

Claude Code lance le run en arrière-plan (nohup) et crée automatiquement la session tmux **zeta-progress** avec le dashboard.

**Étape 2 — Dans un vrai terminal (pas le terminal VSCode) :**

```
tmux attach -t zeta-progress
```

■■ Attention : ne pas confondre avec la session **zeta-cluster** (l'ancien moniteur 4 panneaux htop). Les deux coexistent sans conflit.

**Touches du dashboard :**

| Touche | Action |
|---|---|
| q | Quitter le dashboard (le run continue en arrière-plan) |
| d | Afficher le détail par worker (8 workers par machine) |
| Ctrl+B puis D | Se détacher de tmux sans fermer (le dashboard continue) |

# 4. Lecture du dashboard ZETA RUN PROGRESS

## *4.1 Vue au démarrage du run*

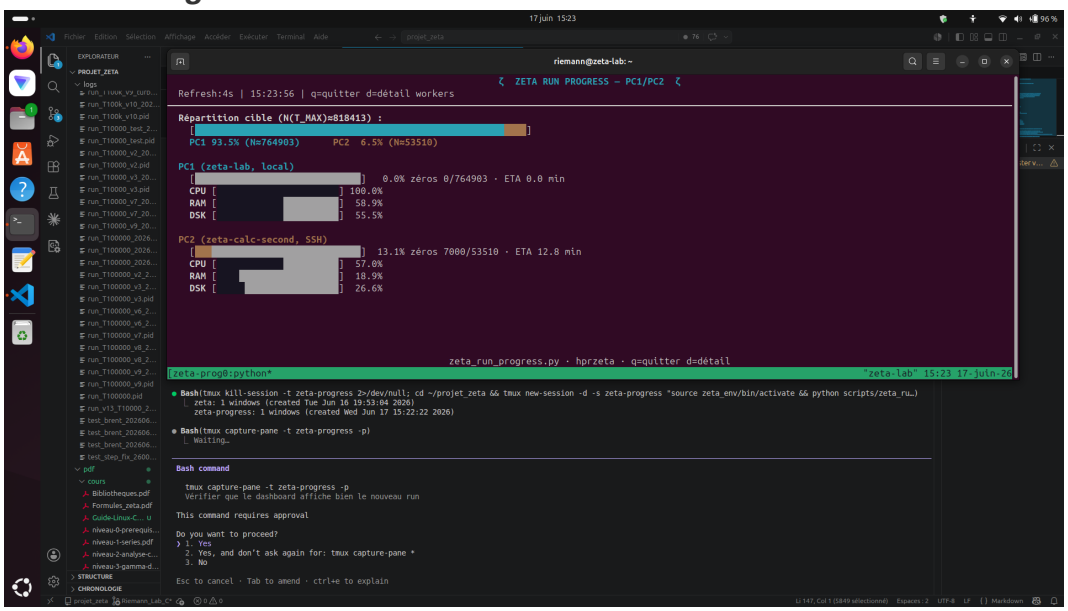



Fig. 1 — Dashboard au démarrage : PC1 0%, PC2 13.1% (7000/53510 zéros, ETA 12.8 min)

Au démarrage, PC2 progresse rapidement car son segment est petit. L'ETA de PC1 est très élevée et non fiable — elle se stabilise après ~10 min quand les workers atteignent leur régime de croisière.

## *4.2 Anatomie du dashboard*

| Élément | Signification |
|---|---|
| Barre cyan (haut) | Part de N(T_MAX) assignée à PC1 (93.5%, N≈764 903) |
| Barre jaune (haut) | Part de N(T_MAX) assignée à PC2 (6.5%, N≈53 510) |
| Barre de progression PC1 | Zéros trouvés / assignés + pourcentage + ETA |
| CPU 100% | PC1 calcule à pleine puissance — normal et attendu |
| CPU 1-6% PC2 en fin | PC2 a terminé sa part, attend PC1 — normal |
| RAM ~62% | Consommation mémoire MPFR/arb — stable |
| DSK 55% | Espace disque utilisé — logs + CSV |
| Refresh:4s | Le dashboard se rafraîchit toutes les 4 secondes |

## *4.3 Dashboard en milieu de run (PC1 ~35%)*

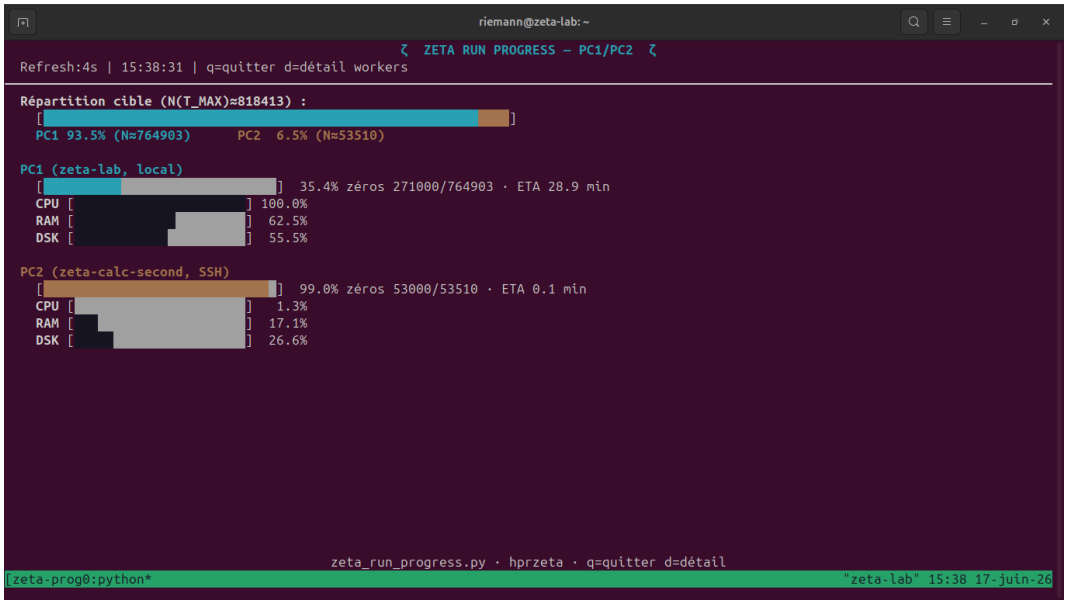



Fig. 2 — 15:38 : PC1 35.4% (271 000/764 903), ETA 28.9 min. PC2 99%, quasi terminé.

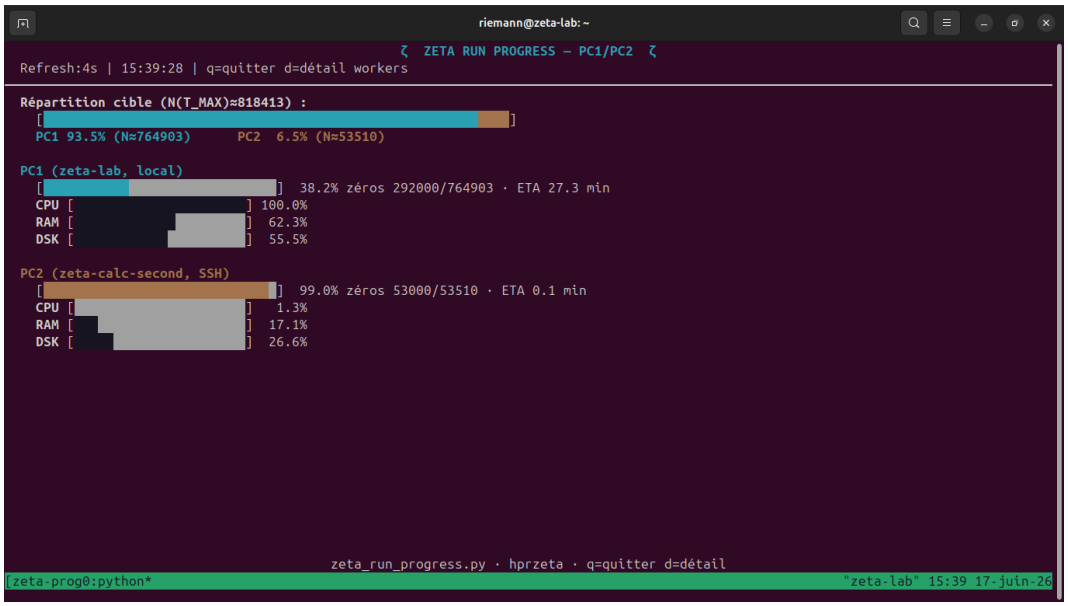



Fig. 3 — 15:39 : PC1 38.2% (292 000/764 903), ETA 27.3 min. Progression régulière.

### *4.4 PC2 terminé — PC1 continue seul*

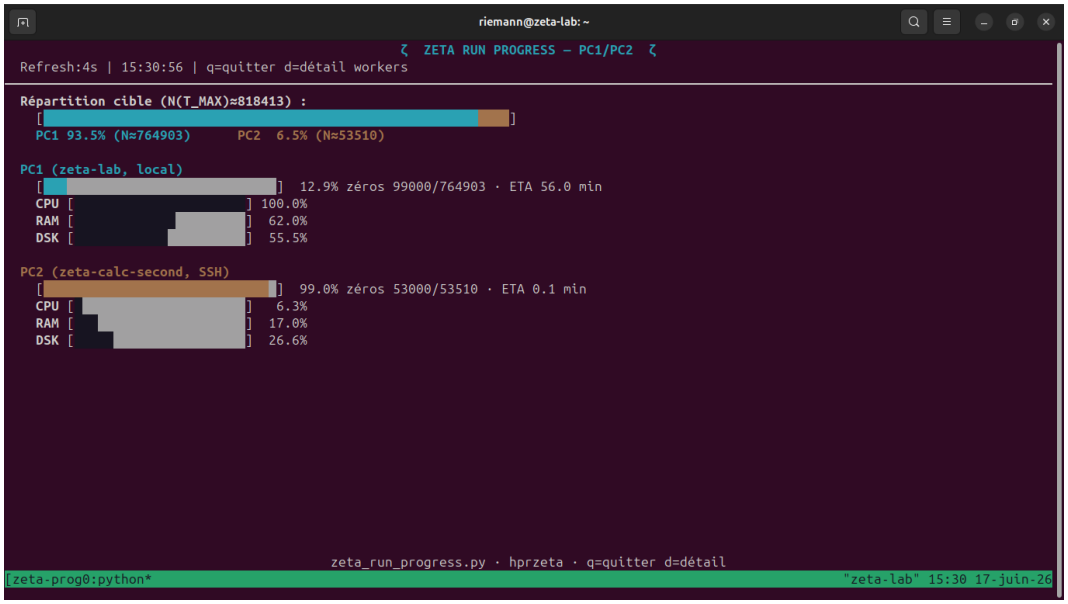



Fig. 4 — PC2 à 99% (53 000/53 510), ETA 0.1 min. CPU PC2 retombé à 6.3% — il attend PC1.

Quand PC2 termine, son CPU retombe à ~1-6% et son ETA affiche 0.0 min. C'est normal — PC2 est en veille active, le processus SSH reste ouvert jusqu'à ce que PC1 finisse et que la fusion soit effectuée.

## 5. Surveillance automatique par Claude Code

En parallèle du dashboard tmux, Claude Code surveille le run en faisant des **captures de texte** (tmux capture-pane) toutes les ~10 secondes. C'est son seul moyen de 'voir' la progression sans vrai terminal interactif.

```
tmux capture-pane -t zeta-progress -p

# → retourne le contenu texte du dashboard
```

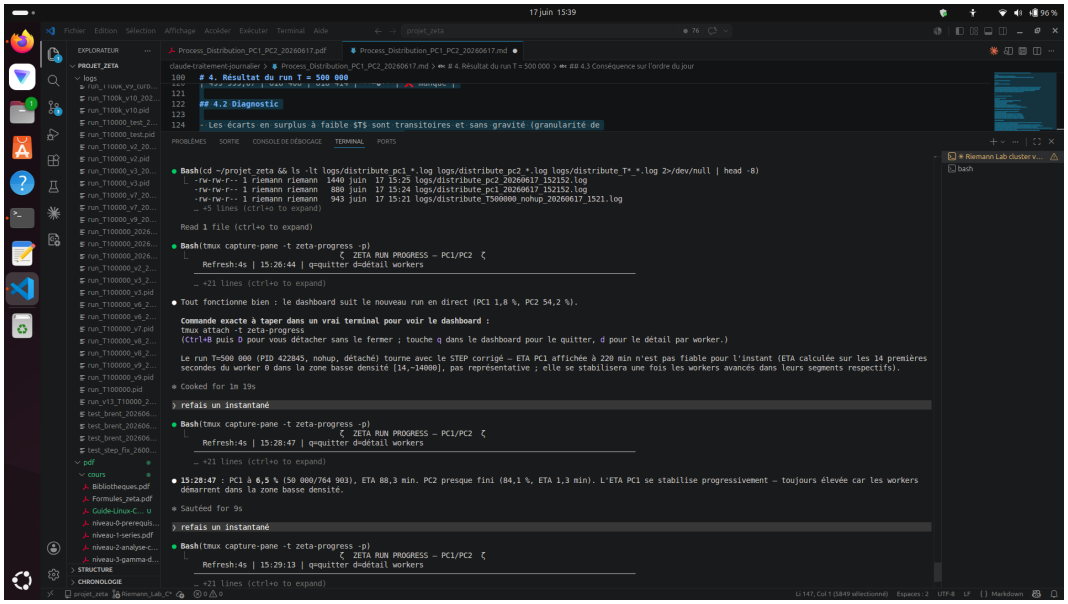


Fig. 5 — Claude Code lit les captures et résume la progression (PC1 6.5%, PC2 84.1%).

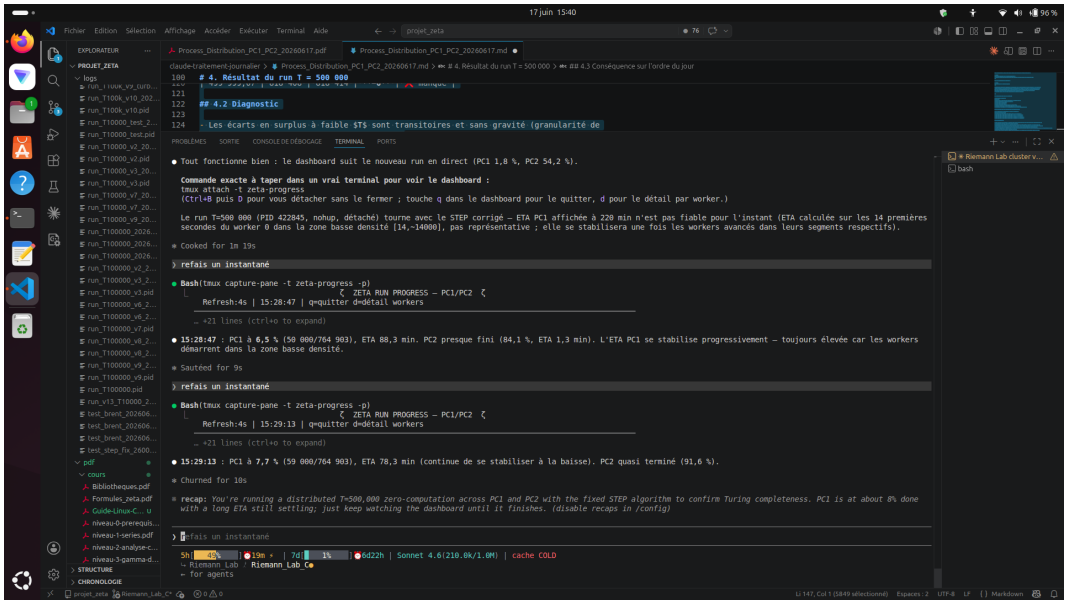


Fig. 6 — Suivi automatique : PC1 7.7%, ETA 78 min. Claude Code commente l'évolution.

**Interprétation des messages de Claude Code :**

| Message Claude Code | Signification |
|---|---|
| refais un instantané | Claude Code relit le dashboard pour mettre à jour ses données |
| ETA PC1 élevée (>100 min) | Normal au début — les workers sont dans la zone basse densité [14,14000] |

| | |
|---|---|
| ETA se stabilise à la baisse | Les workers ont atteint leur segment de croisière — ETA fiable |
| PC2 quasi terminé (91-99%) | Normal — PC2 a peu de zéros à calculer (6.5% du total) |

## 6. Les logs — où et comment les lire

Tous les logs sont dans **~/projet_zeta/logs/**. Ils sont visibles dans l'explorateur VSCode à gauche.

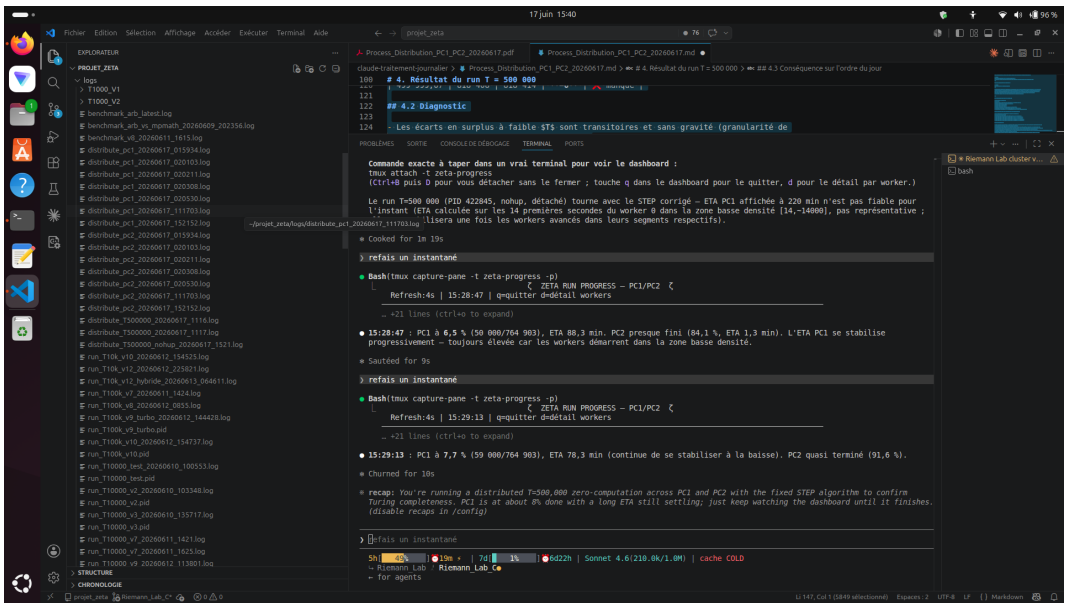


Fig. 7 — Explorateur VSCode : dossier logs/ avec les fichiers distribute_pc1_*, distribute_pc2_*, distribute_T*.

**Fichiers de log générés par zeta_distribute.py :**

| Fichier | Contenu |
|---|---|
| distribute_T500000_nohup_*.log | Log maître : répartition, pivot, statut global |
| distribute_pc1_*.log | Zéros calculés par PC1 (ligne par ligne avec timestamp) |
| distribute_pc2_*.log | Zéros calculés par PC2 (récupéré par SSH) |

**Commandes utiles pour lire les logs :**

```
# Voir les logs les plus récents

ls -lt ~/projet_zeta/logs/ | head -10

# Lire le résultat final du run

tail -30 ~/projet_zeta/logs/distribute_T500000_nohup_*.log

# Suivre PC1 en temps réel (pendant le run)

tail -f ~/projet_zeta/logs/distribute_pc1_*.log

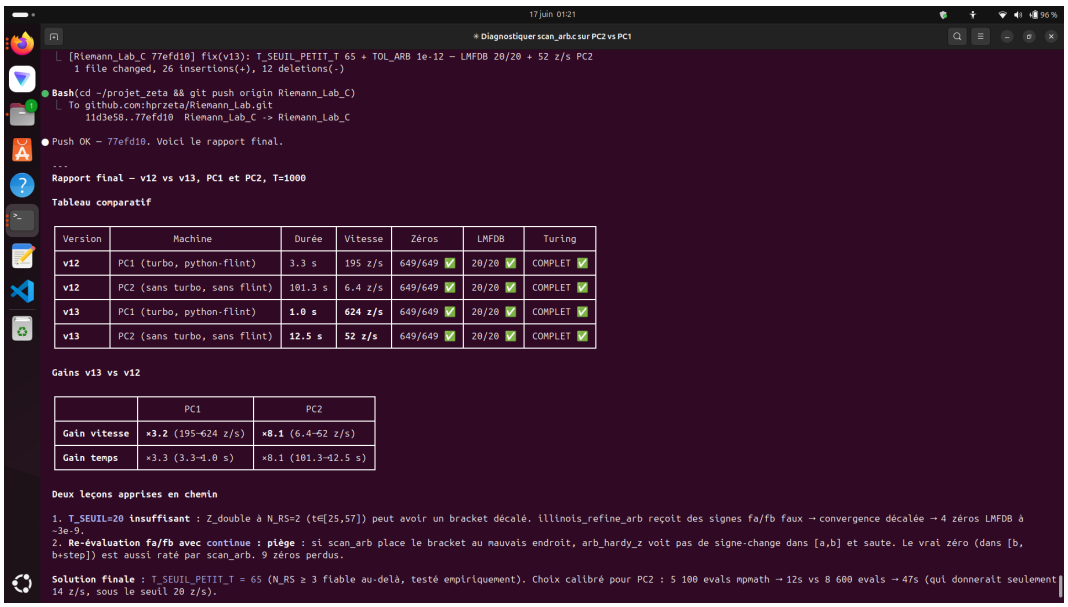
# Compter les zéros trouvés par PC1

wc -l ~/projet_zeta/logs/distribute_pc1_*.log
```

## 7. Interprétation des résultats Turing-Backlund

À la fin du run, la validation **Turing-Backlund** compte le nombre exact de zéros dans [14, T] et compare avec la formule N(T). Un run est valide si et seulement si l'écart est 0 à chaque point de contrôle.

**Exemple de rapport final (T=100 000, v13, PC1+PC2) :**



[Riemann_Lab_C 77efd10] fix(v13): T_SEUIL_PETIT_T 65 + TOL_ARB 1e-12 — LMFDB 20/20 + 52 z/s PC2
1 file changed, 26 insertions(+), 12 deletions(-)

Bash(cd ~/projet_zeta && git push origin Riemann_Lab_C)
To github.com:hprzeta/Riemann_Lab.git
11d3e58..77efd10 Riemann_Lab_C -> Riemann_Lab_C

Push OK — 77efd10. Voici le rapport final.

---

**Rapport final — v12 vs v13, PC1 et PC2, T=1000**

**Tableau comparatif**

| Version | Machine | Durée | Vitesse | Zéros | LMFDB | Turing |
|---|---|---|---|---|---|---|
| **v12** | PC1 (turbo, python-flint) | 3.3 s | 195 z/s | 649/649 ✅ | 20/20 ✅ | COMPLET ✅ |
| **v12** | PC2 (sans turbo, sans flint) | 101.3 s | 6.4 z/s | 649/649 ✅ | 20/20 ✅ | COMPLET ✅ |
| **v13** | PC1 (turbo, python-flint) | **1.0 s** | **624 z/s** | 649/649 ✅ | 20/20 ✅ | COMPLET ✅ |
| **v13** | PC2 (sans turbo, sans flint) | **12.5 s** | **52 z/s** | 649/649 ✅ | 20/20 ✅ | COMPLET ✅ |

**Gains v13 vs v12**

| | PC1 | PC2 |
|---|---|---|
| **Gain vitesse** | **×3.2** (195→624 z/s) | **×8.1** (6.4→62 z/s) |
| **Gain temps** | ×3.3 (3.3→1.0 s) | ×8.1 (101.3→12.5 s) |

**Deux leçons apprises en chemin**

1. `T_SEUIL=20` **insuffisant** : Z_double à N_RS=2 (t∈[25,57]) peut avoir un bracket décalé. illinois_refine_arb reçoit des signes fa/fb faux → convergence décalée → 4 zéros LMFDB à ~3e-9.
2. **Re-évaluation fa/fb avec** `continue` **: piège** : si scan_arb place le bracket au mauvais endroit, arb_hardy_z voit pas de signe-change dans [a,b] et saute. Le vrai zéro (dans [b, b+step]) est aussi raté par scan_arb. 9 zéros perdus.

**Solution finale** : `T_SEUIL_PETIT_T = 65` (N_RS ≥ 3 fiable au-delà, testé empiriquement). Choix calibré pour PC2 : 5 100 evals mpmath → 12s vs 8 600 evals → 47s (qui donnerait seulement 14 z/s, sous le seuil 20 z/s).

Fig. 8 — Rapport T=100k : 138 069 zéros, Turing COMPLET, 0 manquant.

**Tableau de validation Turing (exemple T=500k) :**

| T de contrôle | Calculés | Attendus | Écart | Statut |
|---|---|---|---|---|
| 62 651 | 81 840 | 81 839 | +1 | 🟨🟨 surplus (normal) |
| 142 387 | 204 602 | 204 600 | +2 | 🟨🟨 surplus (normal) |
| 266 309 | 409 204 | 409 202 | +2 | 🟨🟨 surplus (normal) |
| 384 797 | 613 806 | 613 810 | –4 | 🟥 MANQUE (bug STEP) |
| 499 999 | 818 408 | 818 414 | –6 | 🟥 INCOMPLET |

**Surplus en début de plage** (+1, +2) : normal, c'est la granularité de la fenêtre Turing (terme S(T) oscillant). Pas une vraie erreur.
**Manque en fin de plage** (–4, –6) : bug réel — causé ici par le STEP adaptatif trop grand à grand T (corrigé en commit 30d05ee).

## 8. Le bug du STEP adaptatif — diagnostic et correctif

**Cause du problème :**
Le STEP de balayage (pas entre deux évaluations de Z(t)) était fixé à **0.010**, calibré pour T ≤ 100 000. À T = 500 000, la densité de zéros est plus élevée : deux zéros peuvent se trouver dans le même pas → signes identiques → paire invisible.

```
# Ancien STEP (v12, fixe)
STEP = 0.010 # calibré pour T=100k

# Nouveau STEP (v13, adaptatif)
STEP = 0.00497 # ≈ 0.010 / sqrt(2) — deux fois plus fin à grand T
# Formule : STEP = min(0.010, 0.5 / sqrt(N(T)/T))
```

**Validation du correctif :**
Test ciblé sur [260 000, 390 000] avec le nouveau STEP : **224 421 zéros** vs 224 420 anciens → **1 zéro récupéré** (t=273193.66), 0 régression. Tous les anciens zéros retrouvés, strict sur-ensemble.

```
# Commande de validation
python src/calculs/optimisation/compute_zeros_v13.py \
--t_min 260000 --t_max 390000 \
2>&1 | tail -5
```

## 9. Pourquoi 93% / 6% et non 50% / 50% ?

La répartition n'est pas liée à une préférence mais au **rapport de vitesse réel** entre les deux machines. Pour que PC1 et PC2 finissent en même temps :

$$N_{PC1} / V_{PC1} = N_{PC2} / V_{PC2} \rightarrow N_{PC1} / N_{PC2} = V_{PC1} / V_{PC2} = 624 / 52 \approx 12$$

**Facteurs qui expliquent l'écart de vitesse ×12 :**

| Facteur | PC1 (zeta-lab) | PC2 (zeta-calc-second) |
|---|---|---|
| CPU | i7 4 cœurs 3.4 GHz | E8400 2 cœurs 3.0 GHz |
| Turbo | ■ activé (governor perf) | ■ non configuré |
| python-flint | ■ natif Ubuntu | ■ installé 17/06 (pip) |
| libflint-arb | ■ Ubuntu optimisé | ■■ Debian (différences) |
| Workers | 8 workers parallèles | 2 workers parallèles |
| Overhead | Calcul local | SSH + scp réseau |

■ **Perspective v14 :** compiler les modules arb correctement sur PC2 (libflint-arb Debian) et ajouter un script zeta_turbo_on.sh pour PC2 pourrait rapprocher PC2 de 150-200 z/s, réduisant la répartition à ~75%/25%.

## 10. Tableau de performances — v12 → v13

| Configuration | Vitesse | Zéros T=100k | Statut |
|---|---|---|---|
| PC1 seul v12 | 195 z/s | 138 069 | ■ Turing COMPLET |
| PC1 seul v13 | 634 z/s | 138 069 | ■ ×3.2 vs v12 |
| PC2 seul v12 | 6.4 z/s | 138 069 | ■ (sans python-flint) |
| PC2 seul v13 | 52 z/s | 138 069 | ■ (avec python-flint) |
| PC1+PC2 v13 T=10 000 | 350 z/s | 10 142 | ■ Turing COMPLET |
| PC1+PC2 v13 T=100 000 | 270 z/s | 138 069 | ■ Turing COMPLET |
| PC1+PC2 v13 T=500 000* | ~350 z/s | 818 408 | ■ en cours (STEP corrigé) |

* Run en cours au moment de la rédaction de ce guide.

## 11. Récapitulatif — en un coup d'œil

| Action | Commande / Lieu |
|---|---|
| Lancer le run distribué | Prompt dans Claude Code → zeta_distribute.py 500000 |
| Voir le dashboard | Terminal : tmux attach -t zeta-progress |
| Se détacher sans tuer | Ctrl+B puis D |
| Quitter le dashboard | Touche q (le run continue) |
| Détail par worker | Touche d |
| Voir les logs | ls -lt ~/projet_zeta/logs/ \| head -10 |
| Lire le résultat final | tail -30 logs/distribute_T500000_nohup_*.log |
| Interpréter Turing | Écart = 0 → COMPLET ■ \| Écart < 0 → INCOMPLET ■ |